HLD

Notification

https://docs.google.com/document/d/1DclyR9bGLk0TIGuE6DNzUZKgzqzmjx3fzvpin2IoUYg/view?tab=t.0

WHAT WILL HAPPEN IF SYSTEM BREAKDOWN , THEN HOW WILL THE NOTIFICATION BE SENT ONCE THE SYSTEM IS BACK ONLINE

How much time will it take to send notification to 200M users?

what about billions of user will it work in query, what about performance

Will our service store it in the Elastic search and retry ?

Why would we consider 1B users when we applied 80/20 to send Notifications to 20M? Am I misunderstanding?

the user list of scaler will have to be kept synching everyday ... how will that be handled ?

What technology we can implement to introduce AI based architecture into this concept and make some automation based system

How can we predict the approximate amount of Users that will increase suddenly at peak hours?

Firebase Crashlytics tool - it is providing 50k notification use for

How to scale 0 to 1 M

How to scale 0 to 1 million request

1. Initially single server
2. Application & DB Separation
3. Load Balancer & App Servers
4. Database Replication
    a. Database Replication to handle system failures
5. Cache Integration
6. CDN (Content Delivery Network)
    a. Using cache and CDN to improve performance and reduce database calls
    b. CDN is a solution to reduce latency for global users
    c. CDN enables advanced technology with features to optimize load on DB and increase security.
7. Data Center Scaling
    a.
8. Messaging Queues
    a. Messaging queue helps in asynchronous processing of heavy load operations.
    b. Fanout, Topic, and Messaging Queue are important concepts in message exchange
9. Database Sharding
    a. Horizontal and vertical sharding are methods of dividing data in a scalable system.
10.

Active passive architecture

Active Active architecture

Saga Design pattern

Strangler design pattern - use for refactoring to microservices

CQRS Design pattern

OSI Model layer & types

Network communication protocol

1. HTTP
2. TCP
3. IP
4. UDP
5. FTP
6. SMTP (IMAP, POP)
7. Peers to peers model (web-sockets)
8. client server model

Latency & throughput

CAP theorem

Performance vs scalability

1. Vertical scaling
2. Horizontal scaling

Decomposition

Monolithic disadvantage

1. overloaded IDE
2. Scaling is too much hard

What are the advantages and disadvantages of microservices?

Disadvantage

1. latency can be increase if services doesn't design properly /decompose
2. Transaction management little bit difficult if any one services has failed

ACID properties of database(Transactional properties)

Different -2 phase of microservices

1. decomposition - decompose by business capability & decompose by subdomain
2. database -
3. communication
4. integration

Decomposition pattern - divide by below way

1. by business or functionality capability pattern
2. by subdomain pattern (DDD - Domain Driven Design)

Example -> Order management is a microservices (according)
above service divided multiple way (sub domain) - Order placing service & Order tracking services

Data management in microservice

1. Database for each individual service
2. shared database

SAGA (Sequence of local txn) Design pattern resolve problem if we are using database per service then how can we manage Transactional(ACID Properties) for all database because Transactional work for one DB if any one order touching all database then will create local db for all database via use txn(ACID properties)

CQRS design pattern resolve for querying for multiple database just assume if you are using database per service then how can you resolve query issues for multiple database

Why are we not accessing one service's database?
because -> avoid unintentional coupling between services

SAGA Design pattern is two types
1. choreography - for one is drawback for choreography -> cyclic dependency will create
2. Orchestration

CQRS - DB Trigger insert data into database

Database replication - important
1. **Master sloup** concept for handle failure scenario

if any **replica DB** failed then replica DB can take care - replica DB is only for read operation
if Master DB failed then replica DB converted in master DB - Master DB is only for write operation

Cache mesh implement before database call and cache interact with application layer and
Write in cache from db (TTL) - **important**
CDN (content delivery network) - CDN does caching but All those who do caching are not CDN. - **Important**
CDN - caching of static data like html, video, css and other
CDN will create CDN Nodes and avoid DDOS attack
client directly will go with cdn without hitting server via load balancer
Asynchronous nature is way for fast process - messaging Queqe - like kafka, Rabbit MQ
Messaging Queue - Important - how can we scale our application use via MQ
Database scaling is two ways - **important**
1. vertical scaling - vertical has limit - means CPU and RAM increase is a limit
2. horizontal scaling - increase nodes and sharding

Sharding is also two types - **important**
1. horizontal sharding- ?
2. Vertical sharding-

Denormalization for avoid joining
if making tree sharding then resolve via Consistent hashing

Consistent Hashing - Horizontal sharding - **Important** (for scaling our application) - its use for equally divided for handling load in our server or database

Arbitrary length - arbitrary length value is converted into fix length value
Hash function
mod hashing - it is fix size for storing in index
What is the problem with hashing?
What is the problem with consistent hashing?
What are the problems with static hashing?
Bucket Overflow: If a bucket becomes full and cannot accommodate more elements, it leads to bucket overflow. Static hashing techniques need to handle bucket overflow efficiently, either by using overflow chains or by probing for empty buckets.

What is rebalancing? - for million of DB entries- its consistent hashing is solution if we have multiple node

Disadvantage of consistent hashing ?
1. replicate at virtual object points

https://www.withouttheloop.com/articles/2014-09-30-postgresql-nosql/
https://www.keycdn.com/support/anycast
https://www.cloudflare.com/en-gb/learning/cdn/glossary/anycast-network/
https://relevant.software/blog/microservices-database-management/

BOE Estimation

Back of the Envelope Estimation

Drive our decision for system design

What is the back of the envelope estimation? Why does it need this?
8st
Consideration
1. Rough (T-shirt size) estimation
2. do not spend much time
3. keep the assumption value simple

cheat sheet of Table

3 zero - Thousand - kb
6 zero - million - mb
9 zero - billion - gb
12 zero - trillion - tb
15 zero - qualion - pb

char -> 2bytes
long/double -> 8 bytes
images(average) -> 300 kb

there are three things are required
1. no of servers
2. RAM
3. storage capacity

Tradeoff (CAP) ->

Traffic estimation-

Total user->

Cheat sheet

3 zero	Thousand	Kb
6 zero	million	mb
9 zero	billion	gb
12 zero	trillion	tb

15 zero	quadrillion	pb

Char - 2bytes
long/double - 8bytes
images(average) - 300kb

There are three things required
1. No of servers
2. RAM
3. Storage capacity

Tradeoff (CAP)

Traffic Estimation -
Total user: - 1 billion
Daily active user - 25% of total user = 250 million user

Read & write operation -
5 + 2 = 7 query of read and write operation

250 million user(DAU) * 7/86400 sec = approximate 18k query per second

Storage assumption:

Each user doing 2 post (each post 250 char)
10% of total user uploading images (300kb * 10 billion)

1 post —- 250 char —--- 250 * 2 = 500 byte
2 post —-- 500 * 2 —--- 1000byte = 1 kb

205 million users * 1 kb = 250 GB /day for only post

250 million users (DAU) * 300 kb = 7500 GB ~ 8 TB/day (approx)

5 years = 1825 days ~ 2000 days

(post) 2000 * 250 GB = 500000 GB ~ 500 TB
(image) 2000 * 8 TB = 1600 TB ~ 16 PB

RAM estimation

For Each user => last 5 post putting cache
1 post = 500 byte
5 post = 500 * 5 = 2500 byte (cache)

250 Million (DAV) * 2500 byte (Post) =
250 million (DAV) * 3 Kb = 750 GB (memory GB)

1 machine = 75 GB for in-memory
Means 10 machines are required for 750 GB in-memory

Latency = 95 % = = 500 ms
18 k request per second

1 server has 50 thread then
1 second = 2 (500 ms) request
1 sec server 100 request /second

Then we can calculate server 180 server are required to handle 18k request per second

SQL vs No SQL

SQL vs No SQL

There are four division of SQL (Structured Query language)

1. Structure- Redefined Schema (Table, row and others) and manage
2. Nature- concentrated/centralized
3. Scaling - vertical/horizontal
    a. Vertical - increase RAM/Storage as per vertical use cases (SQL is good)
    b. Horizontal - increasing sharding, not well supported centralized data
4. Property
    a. ACID Properties - its makes sure data integrity and consistency
        i. A - Atomicity
        ii. C - Consistency
        iii. I - Isolation
        iv. D - Durability

NOSQL

We are dividing four division of NOSql (Unstructured data)

1. Structure
2. Nature - data can be distributed
3. Scaling - growing horizontally not vertically
4. Property- its is following BASE and not following ACID property
    a. BA - Basically Available
    b. S - Safe state (state can be changed without any interaction)
    c. E- Eventual consistency

NoSQL has some types DB and it has structure of NoSQL

1. Key-value DB - DynamoDb is Key value DB
2. Document DB- MongoDB
3. Column wise DB -
4. Graph DB - usage - social networking platform & recommended engine, Node and Edge

When to use SQL and NoSQL

SQL	NoSQL
Flexible query functionality	complex query will not support(Basic query search)
Data kind of relational nature	
Data Integrity and you can’t lose consistency	It is following BASE that’s why its chance to lose consistency
Availability	High Availability and high performance & not consistency because distributed nature

What is the difference between Key-value DB and Document DB?

What is the difference between Column wise DB and Document DB?

What is the difference between Graph DB and Column wise DB?

Design Rate Limitor

Design Rate Limiter

DDOS Attack

Attacker will send unwanted multiple times request per second to the server that's why happened **DDOS Attack**
Each server has limited resource like RAM, Storage

That's why come below Algorithm

1. Token bucket
2. Leaking bucket
3. Fixed window counter
4. Sliding window log
5. Sliding window counter

1. **Token bucket**:
    Its has some capacity means hold token
2. Leaking bucket:
    It has a constant request and implemented from queue
3. Fixed window counter:

Idempotency Handling

Idempotency Handling

Idempotency vs concurrency

Concurrency

You have one resource there are multiple use try to access request

Idempotency

Help use to take care of duplicate request

It enable client to safely retry an operation without worrying about the side-effect of operation

By default GET, PUT, DELETE are idempotent

There are two types of idempotent
1. Sequential
2. Parallel

How can it be duplicated?

What is the difference between sequential and parallel?

Sequential	Parallel

Bring the lock for parallel situation

MuteX - acquire and release
Semaphore -
Synchronized -

If you are using multiple server then use cache because cache is much more faster sync

Design High Availability Architecture

Design High Availability Architecture

Can be build that's way

1. Design data Resiliency Architecture
2. Design Architecture to achieve 99.999% availability
3. Design to avoid single point of failure
4. Active passive vs active-active architecture

Resilience - its prevent failure and providing high availability

Always chance to failure in single node (single point of failure- database not working)
Single node means you are doing all these in a single node like LB, application, database and everything there into a single node.

We can solve these thing via multi node

There are two types of multi node

1. Active-passive architecture
2. Active Active architecture

Every company has two data center
Just assume one is data center is mumbai and second one is pune

And only one data center is primary(read and write, Live DB) and other one treated as replica (DR-Disaster recovery data center) DB

Sync up will be happen primary and secondary DB (replica DB)

Reason - Oracle/MySQL/postgres are not multi-master db (write to only primary/live db) and replica DB is only for read only as per balancing load

If any one primary DB has down then we can route traffic disaster recovery data center and we can avoid single point of failure

Active-passive architecture only one directional sync happened from primary to replica Data center

Disadvantage

1. If any thing happens then goes disaster then will be latency issues
2. If any db failure happen then we can route primary replica then will be gap that's was time failure for connect db
3. If any one primary db has too much write operation then it will impact much more primary db because primary db serving write operation only and replica db serving read operation only

Active-Active Architecture

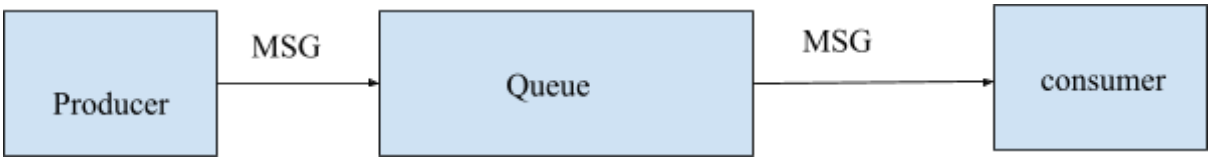
RDBMS does not support multi master write(more than one life we can write & db can write simultaneously)
Active-Active architecture bi-directional sync happened from primary to replica data center
And it will create conflict for write and read operation and its difficult to maintain sync in active-active architecture

But we are utilizing much more resources in active-active architecture

Distributed Messaging Queue

Distributed Messaging Queue (like Kafka, RabbitMQ)

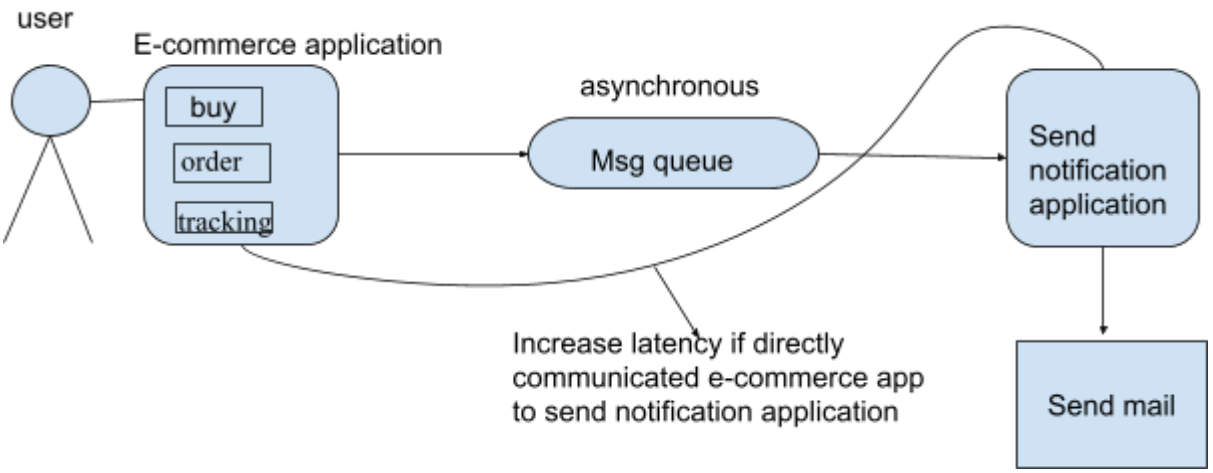
What is a messaging Queue ? **Why is it needed**?



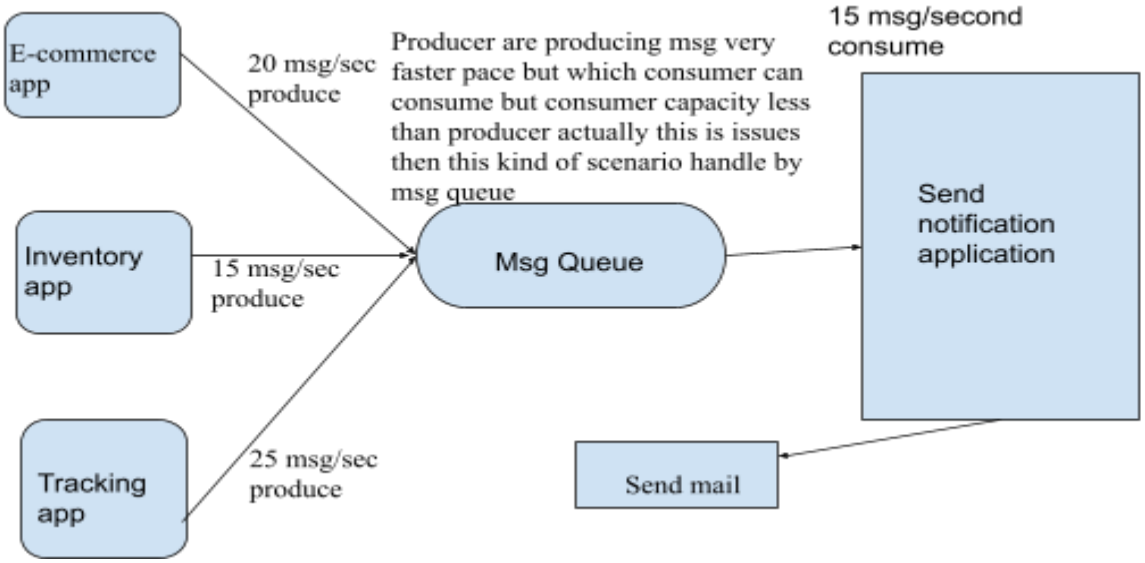
Which is queue then store msg

Usagecase

1. Asynchronous nature
2. Retry capability (if send notification application has down then queue will retry capability)

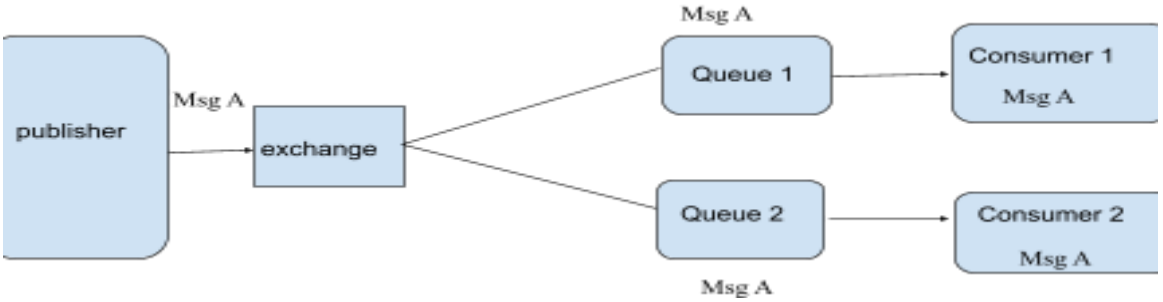


3. Pace matching (msg queue help us tackle from producer and consumer consume as own pace)
   Example:



What is point 2 point?
Whenever a particular msg A put into a message queue then only one consumer consume not other consumer.
Single message only consumed by single consumer

What is the publisher and subscriber pattern?
Broadcast msg as per exchange logic to all queue
In pub/sub, same msg can be process by multiple consumer

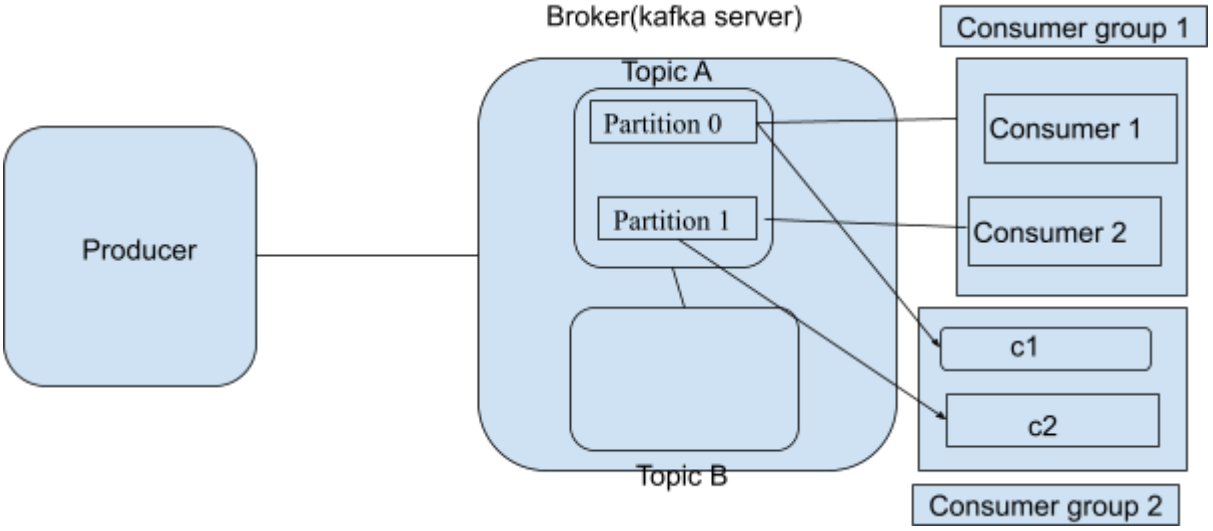


How does the messaging queue work like kafka?

Kafka has so many component
1. Producer
2. Consumer
3. Consumer group
4. Topic
5. Partition
6. Offset
7. Broker (kafka server)
8. Cluster
9. zookeeper

We have Producer and producer talk with broker, broker has a kafka server, broker has topic and it can be many topic
Insider any topic has many partition and inside partition has offset and different different length
Consumer read from partition and data will store from partition
And each consumer part of consumer group means in consumer group has multiple consumer

Now inside the inside particular group logic they don't share, within partition consumer group access different group but within same consumer always read different partition of particular topic



Producer talks to broker broker has a topic and topic has different-2 partition

Group of different machine/node that's called **cluster**
Different -broker how can we interact with each other that's called **zookeeper**

Consumer group read data from partition

Why is offset important ?
Consumer 1 and consumer 2 both are part of same consumer group
Let's two consumer like consumer 1 and consumer 2 and if consumer 1 has goes down then what will be happen
Kafka will pick from consumer 2 and start from where left from consumer 1
Offset is use keep to track what all msg are read and get to read using committing offset msg

Broker with the help of cluster and Cluster is the group of broker
Topic has two partition broker has Topic 1 and partition has zero
Topic is a leader and same partition replica to other broker is called follower

Read and write happened through leader whenever leader goes down then replica take over and it will be become leader
Follower always follow leader means any new msg will comes up then sync both one

What happens when the queue size limit is reached?
We will add different broker because particular machine has limited size
What happens to messages when the queue goes down?
Nothing will be happen then follower will be leader because both will be sync to each other
What happens when the consumer goes down?

If consumer goes down then another consumer take over from same consumer group

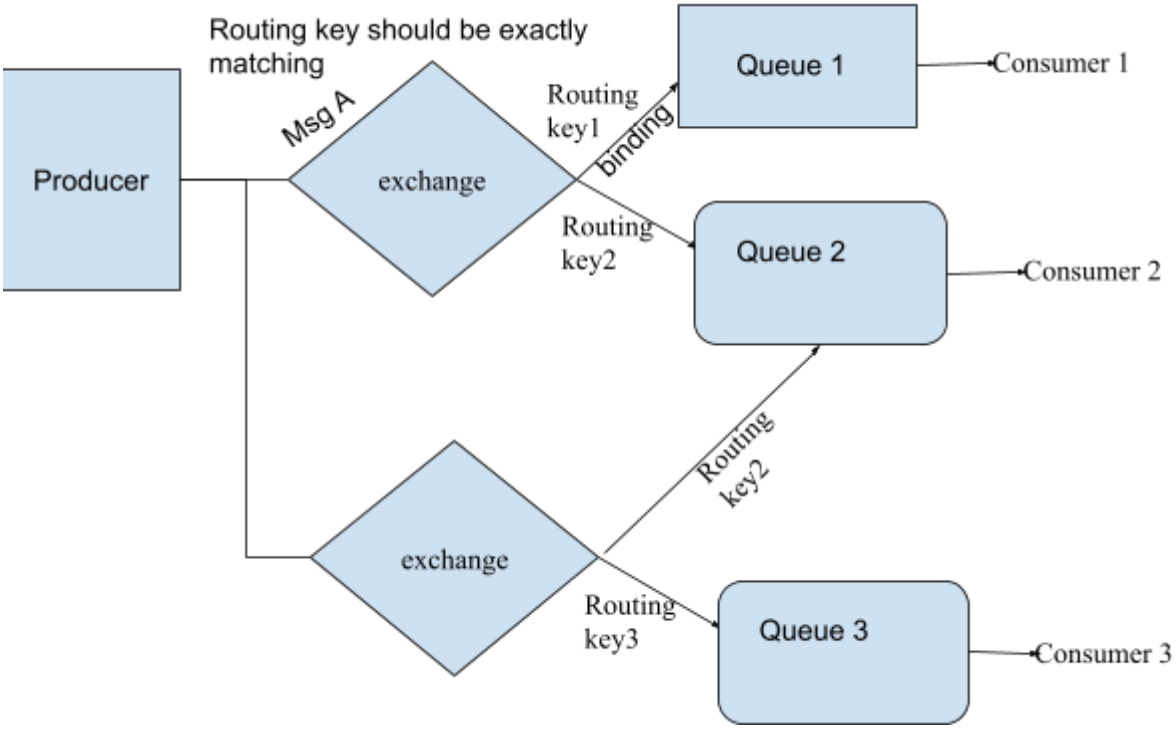
What happens when consumers are not able to process messages?
Every msg needs to retry 3 because this is configuration in the partition if retry limit will be reached and moved into failure queue/dead say for produce for different msg.

How does retry work? **Different ways to retry**?

RabbitMQ	Kafka
Push based approach	Pull based approach (consumer pulling means any new request is there or not

How does a distributed messaging queue work?
What is a dead letter queue?

Rabbit MQ



Exchange

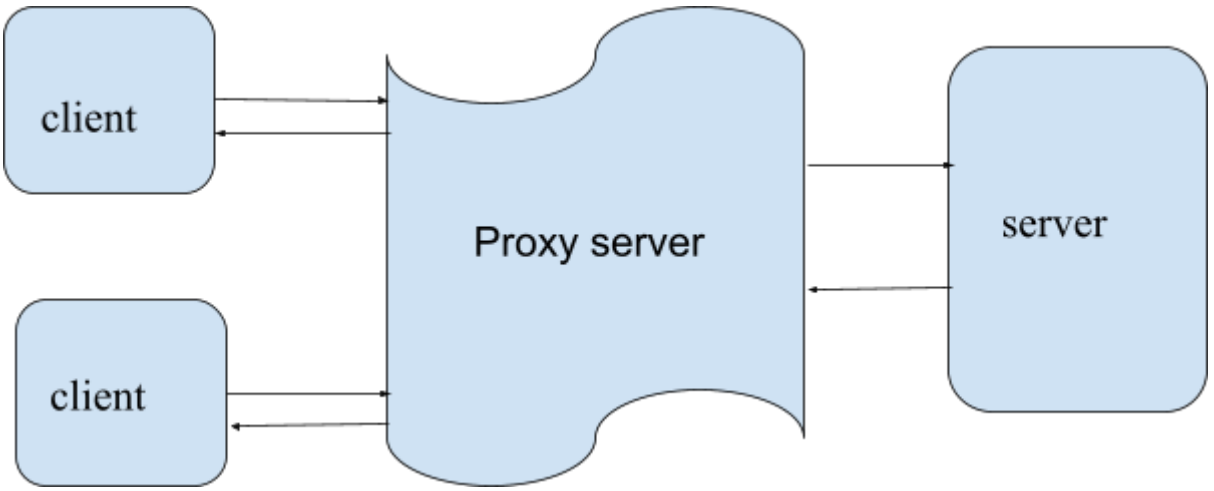
1. Fan out
2. Direct exchange (exactly match msg key1 & routing key) to if match then push to queue 1

3. Topic exchange (wild card)
4. Offset concept is not there in rabbitMQ
5. If any msg not processed or something bug then msg requeue again but kafka retry option if reached then death queue

Proxy server

Proxy Server

What is a Proxy server?
Proxy is taking a request someone of behalf
Proxy can take request more than one client also



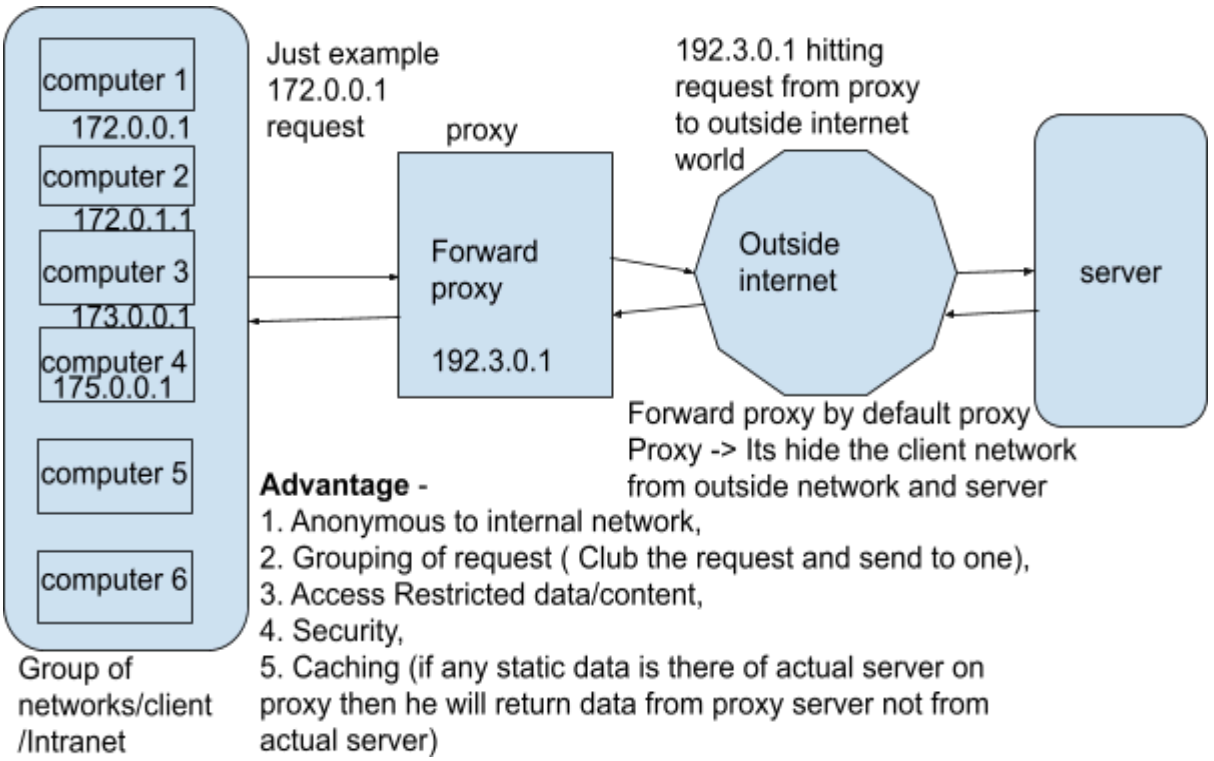
Proxy server is the middle man for client and server

What are its different types?

- ➔ Forward proxy
- ➔ Reverse Proxy

Differ in direction of communication

Forward proxy-



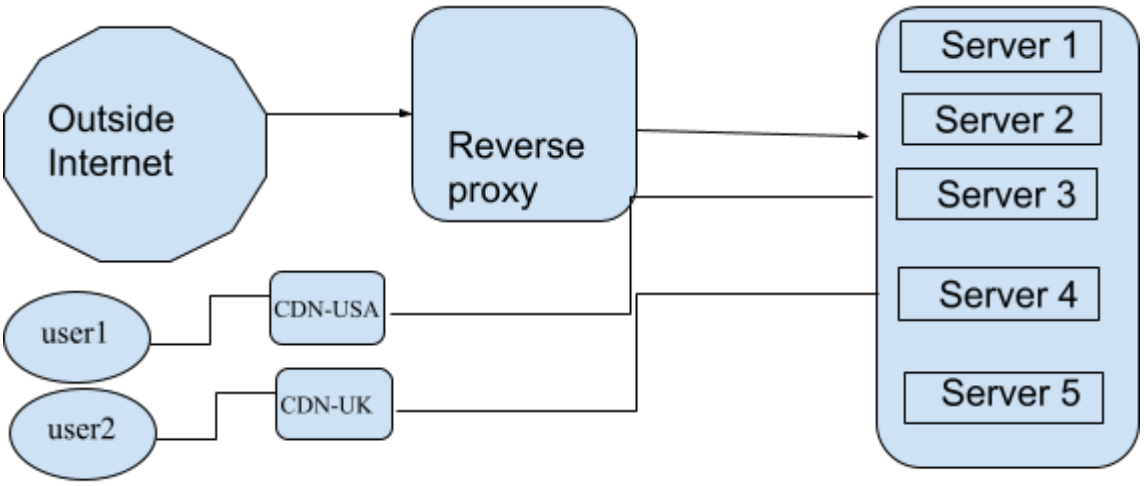
Advantage -
1. Anonymous to internal network,
2. Grouping of request (Club the request and send to one),
3. Access Restricted data/content,
4. Security,
5. Caching (if any static data is there of actual server on proxy then he will return data from proxy server not from actual server)

Disadvantage

1. Application layer (each application level setup proxy)
2.

Reverse Proxy

Its opposite of forward proxy
Means request are coming from internet through reverse proxy to group of server
If any attack will be happen then attacker can attack only reverse proxy not on server
Like DDOS attack



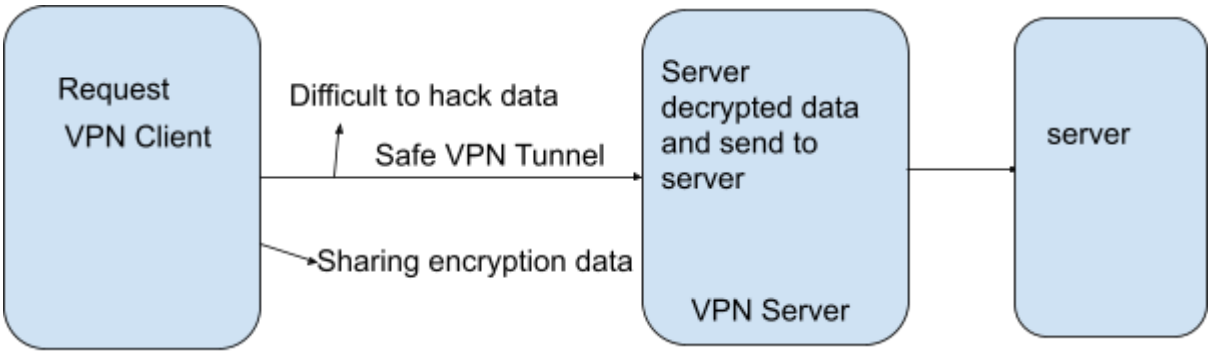
Advantage

1. Security (handle DDOS Attack - if anything will be happen then will not affect on actual server
2. CDN (it the reverse proxy - if any user requesting first goes to cdn if data is not there in cdn then will goes to server)
3. reducing latency
4. Load balancer capability (distributing load on different-2 server)

Proxy vs VPN

Proxy doing bypass server and proxy is doing only masking IP address but Proxy not encrypted data

VPN create a tunnel and doing Encrypted/decrypted data where data flow



Safe VPN tunnel whose flow the data that's called VPN tunnel

Reverse Proxy vs Load Balancer

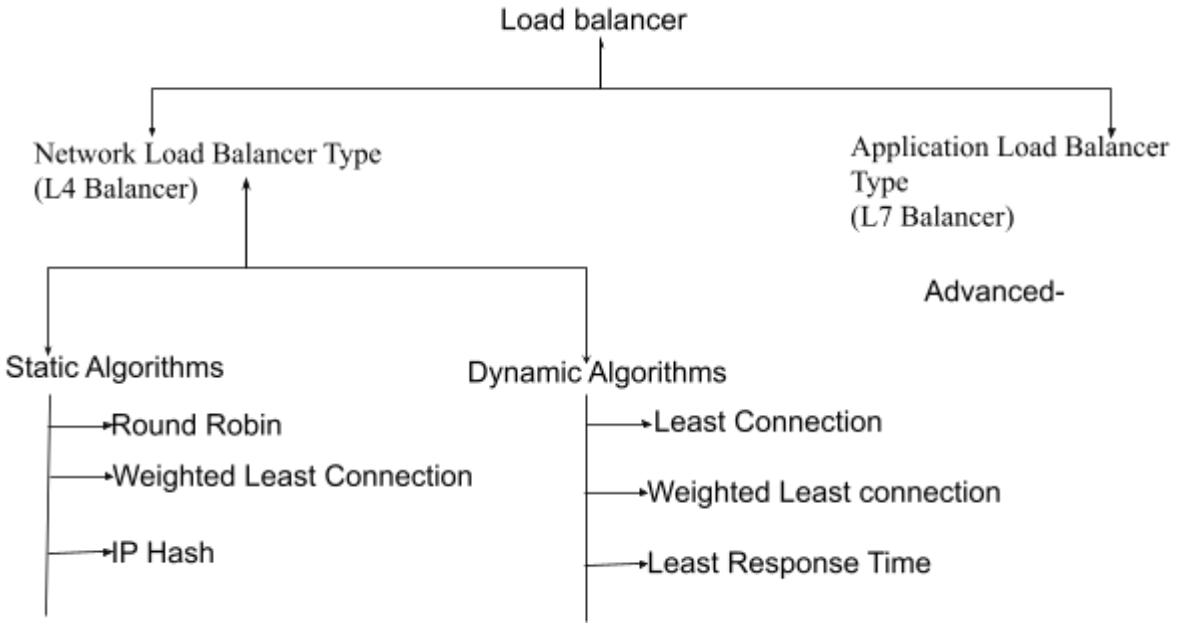
Reverse Proxy has also acted as load balancer but load balancer can't be proxy
Proxy can do anonymity, caching and logging etc. but load balancer no
If you have more than one server then you need load balancer otherwise it is not required

Proxy vs firewall
Firewall has certain holes and each hole has defined rules (packet scanning) like port, ip etc.
And proxy works on application layer
Proxy has acted as firewall but firewall is way work as traditional firewall

Load Balancer & Different Algorithms

Load Balancer & Different Algorithms

Load balancer is used for distributing this traffic appropriately to all the servers.



Network load balancer -

1. Its is Faster-
    a. Because it can have TCP Port, UDP Port, IP Address, source & destination (based on this you can take decision)
2. Ddd

Application Load balancer:

1. It can read your header, session even data and it also read your response, cookies and also have capabilities caching because reading response so that's why we can say it is advance

Static Load balancing

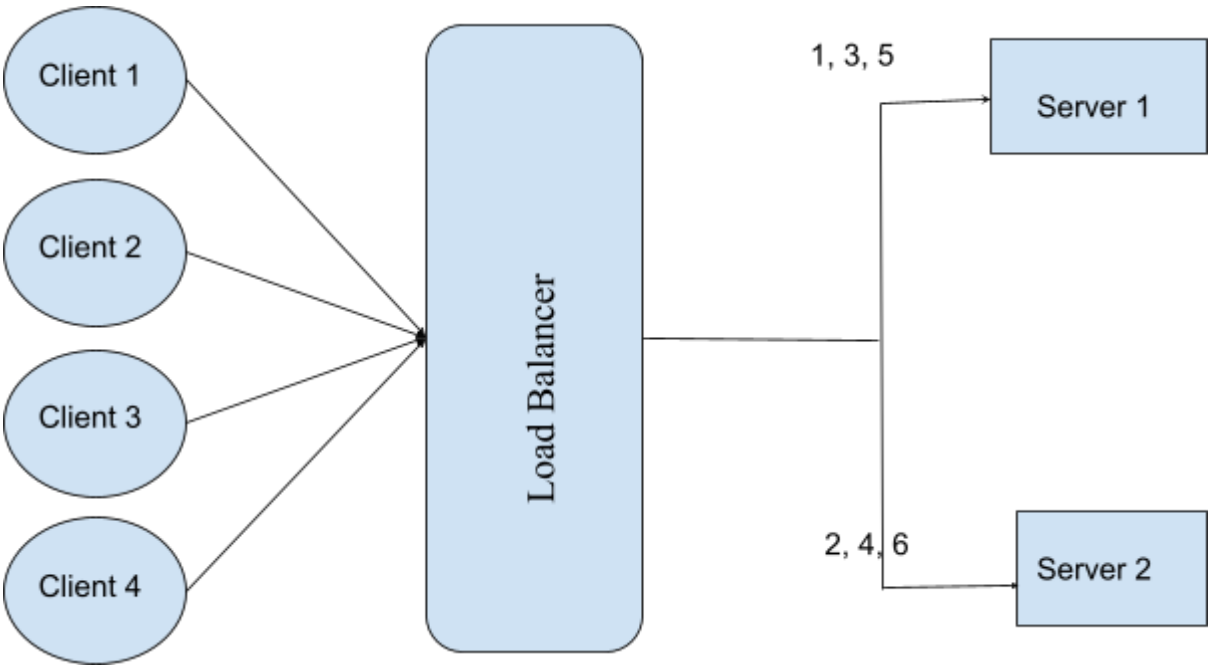
Round Robin:

Advantage:

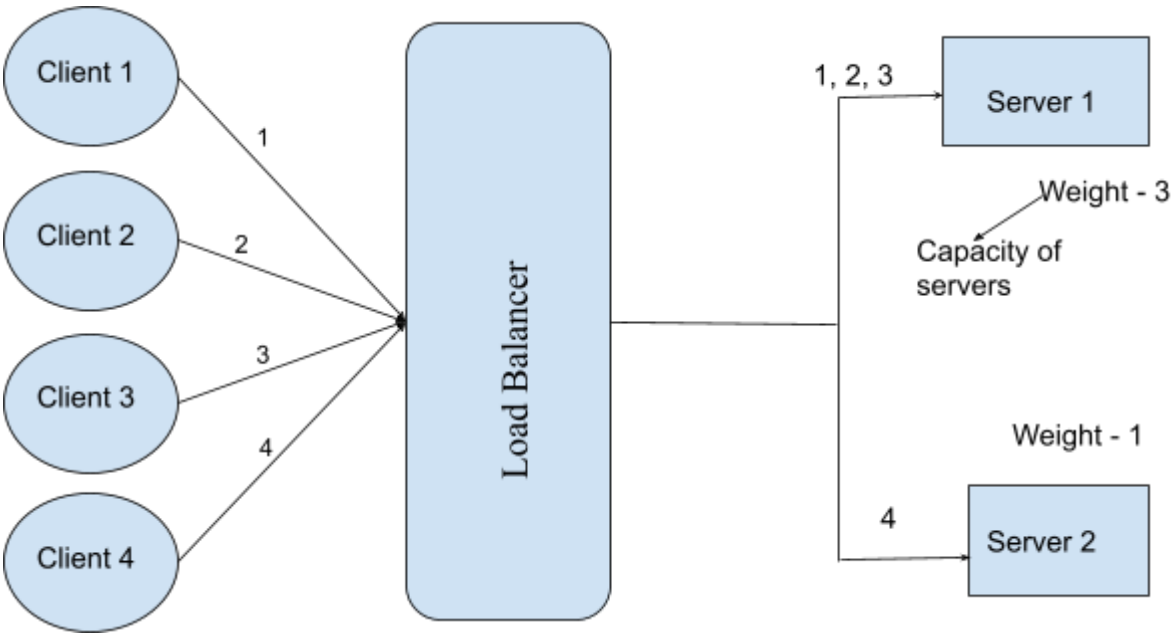
1. Very Easy to implement
2. Equal load distribution to all the servers

Disadvantage

1. One server with high capacity and another with low, both will be treated as same
2. Chance that low capacity server will go down because of overload of requests.



Weighted Round Robin:
Weight represents capacity of server



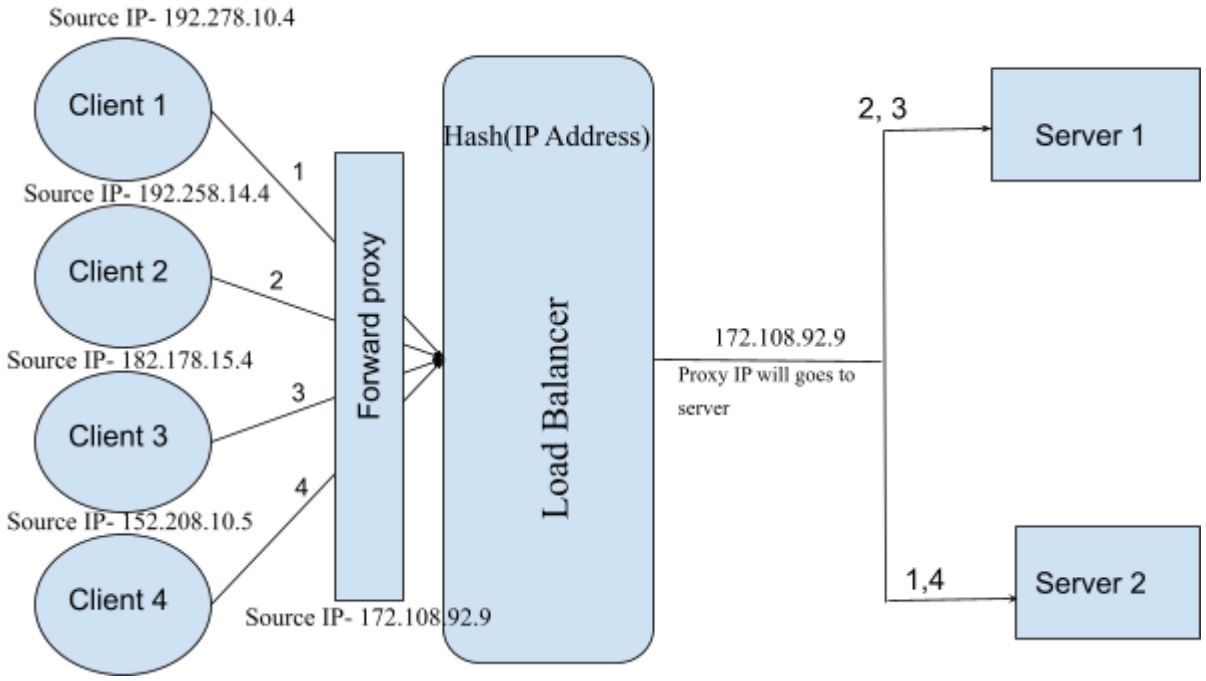
Advantage:

1. Low Capacity server, will get saved from receiving the large no of requests
2. Easy to implement as weight are static, no dynamic computation

Disadvantage:

1. If requests have different processing time, then its possible that low capacity servers get high processing requests and get overburdened.

IP Hash



Advantages:

1. Good for use cases, where the same client needs to connect to the same server.
2. Easy to implement

Disadvantages:

1. If a client's request is coming through proxy then all the clients will have the same source IP address, and this will overload one server.
2. Can not ensure equal distribution.

Dynamic Load Balancing

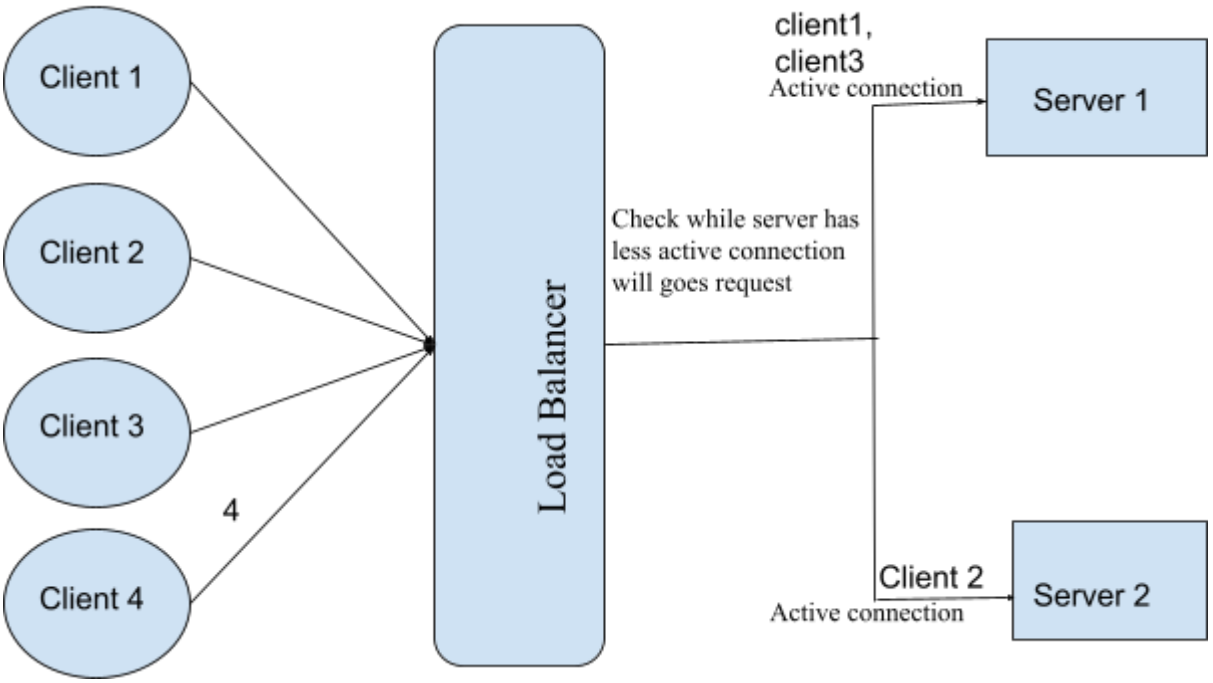
Least connection request

Advantage:

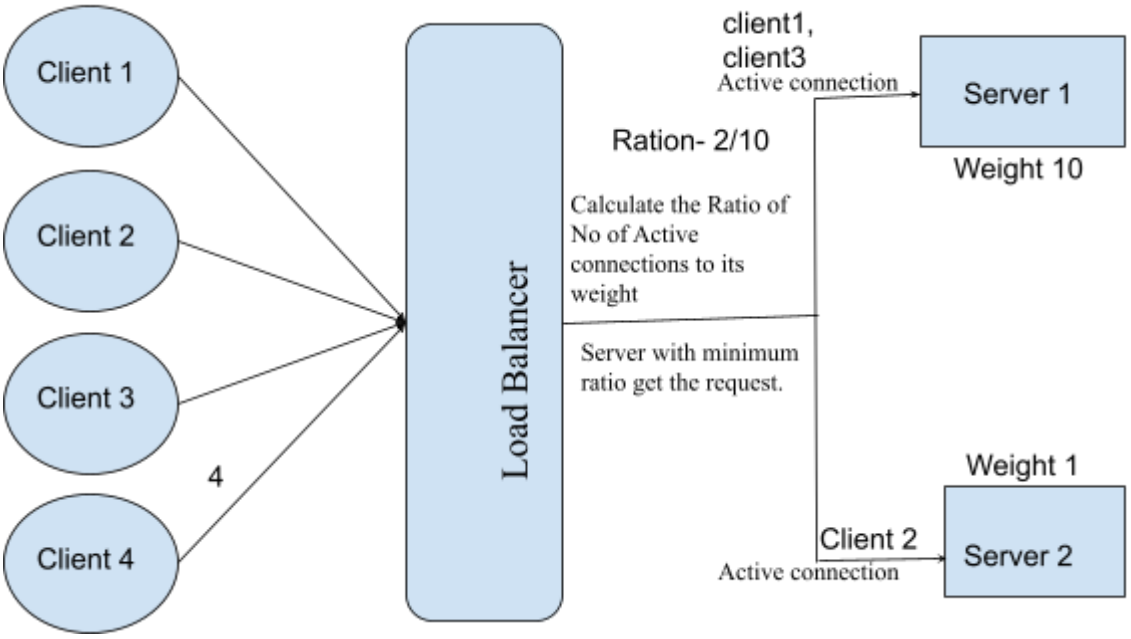
1. Dynamic, as it considers the load on each server, so the chance of overburdening of the server is less when each server has equal capacity.

Disadvantage:

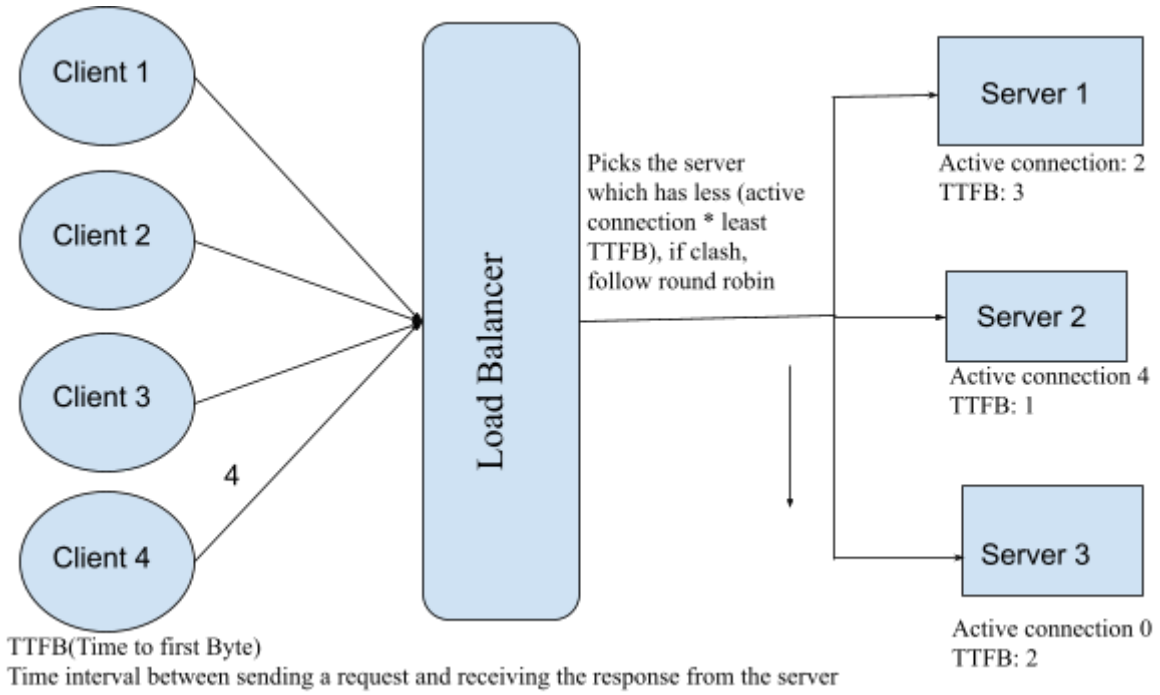
1. TCP connections can be ACTIVE but possibly have no traffic. So purpose is failed
2. No Difference between low capacity and high capacity server. Chance that low capacity server will get burdened.



Weighted Least connection:



Least Response Time:



What is anycast-network?

What is OSI Layer ? and how many times ?

Distributed Cache and Caching Strategies

Distributed Cache and Caching Strategies

What is caching?

What is distributed caching and how is it done?

Caching strategy

1. **Cache aside**
2. **Read-through**
3. **Write around**
4. **Write through**
5. **Write back**

Cache Eviction policies

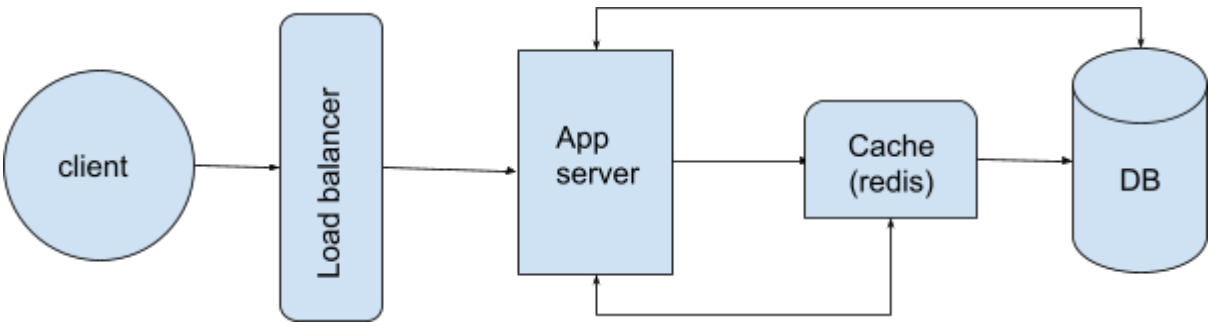
1. **LRU**
2. **FIFO**
3. **LFU etc.**

What is caching?

- Caching is a technique to store frequently used data in a fast access memory rather than accessing data every time from slow access memory.
- This makes our system very fast
- It helps to reduce the latency
- It also helps to achieve the fault tolerance

There are different type of caching present at different layer of the system like

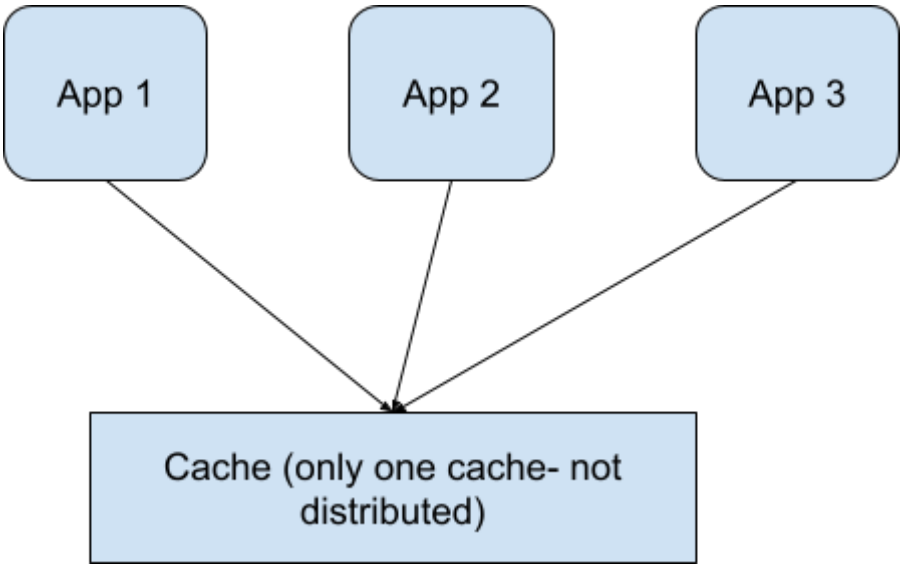
- Client side caching (browser caching)
- CDN (Used to store the static data)
- Load Balancer
- **Server side Application caching**(like Redis etc)



What is slow access memory and fast access memory?

Slow access memory means RDBMS DB,

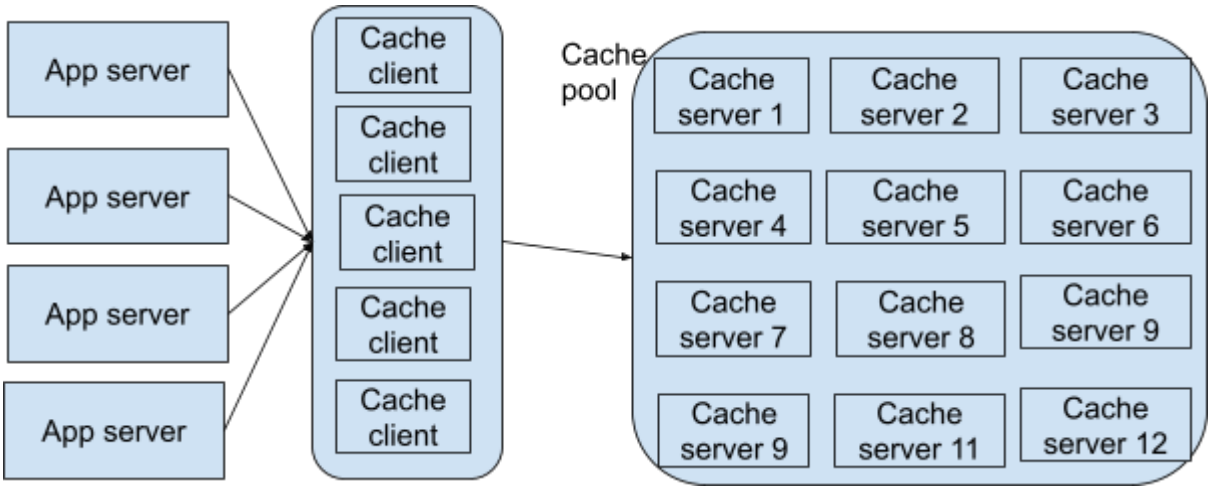
Fast access memory means RAM



limitation of Only cache

1. Single point of failure (if cache failure then caching capability gone)
2. Scalability (one particular time, you can't scale)

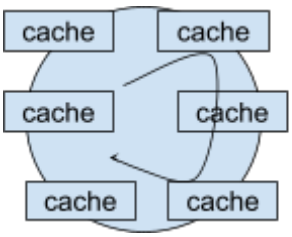
What is distributed caching?



Each app server has own cache client and cache client get one of the cache server from cache pool (where store the data and read the data)

By using **consistent hashing technique** – one of the server to allotted to particular caching server

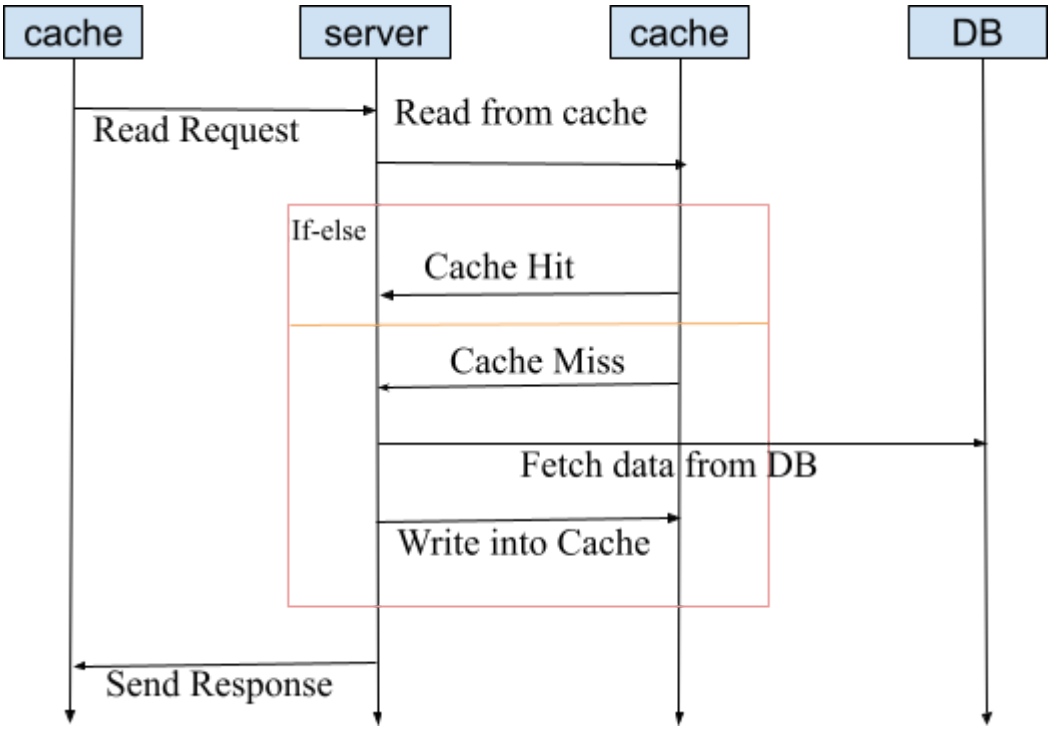
Consistent hashing technique makes a ring and all the cache server will be present there on the ring



Clockwise rotate

What is a Caching strategy?

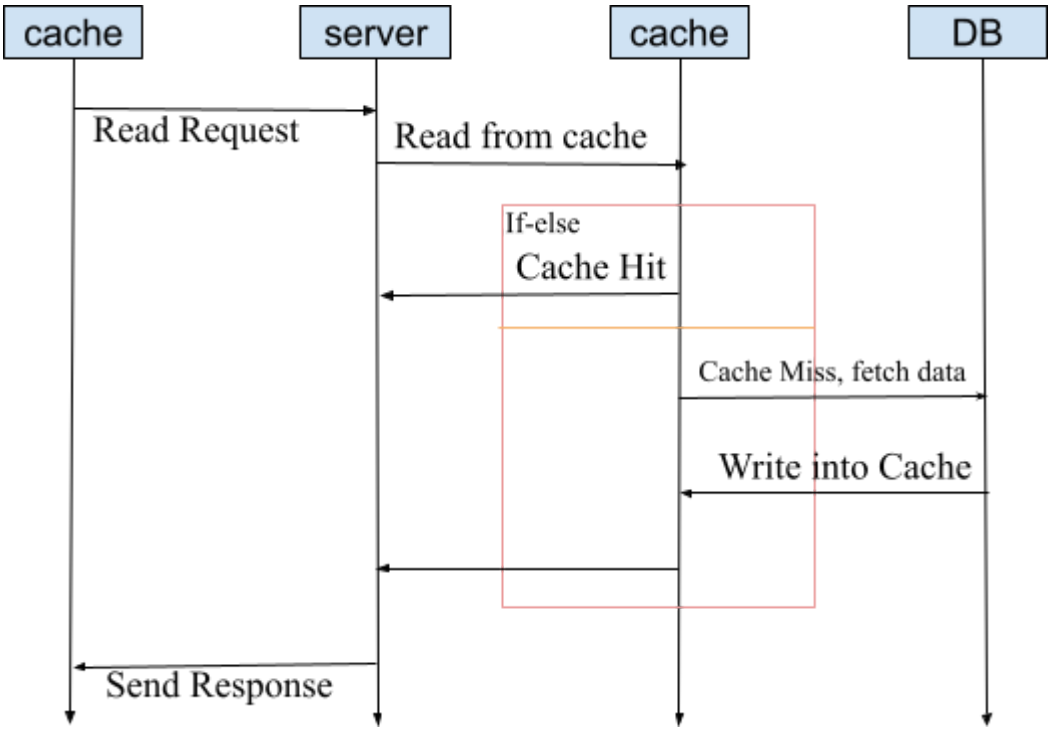
1. **Cache Aside**:
    a. Application first checks the cache.
    b. If data found in cache, its called cache Hit and data is returned to the client
    c. If data is not found in cache, it's called cache Miss. Application fetch the data from DB, store it back to cache and data is returned to the client.



Pros:
1. Good approach for heavy read application
2. Even Cache is down, request will not fail, as it will fetch that data from DB
3. Cache document data structure can be different from the data present in DB.

Cons:
1. For new data read, there will always be cache-misses first. (to resolve this, generally we can pre-heat the cache)
2. Without appropriate caching is not used during write operation. There is a chance of inconsistency between cache and DB.(**inconsistency**)

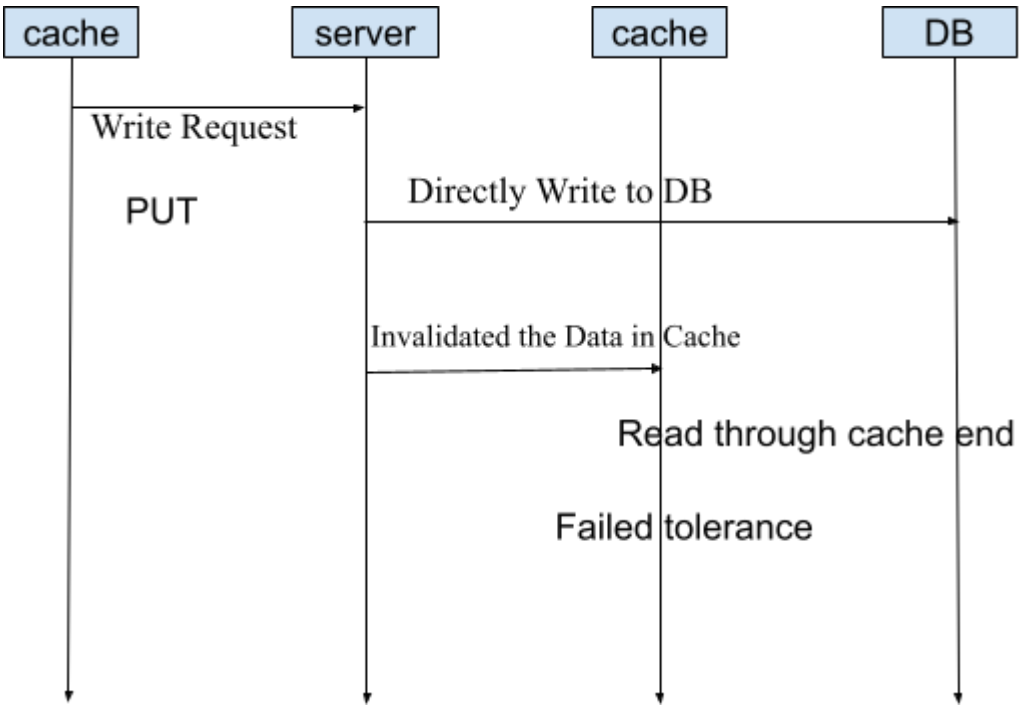
2. **Read through Cache**:
    a. Application first checks the cache.
    b. If data is found in cache. It's called cache Hit and data is returned to the client.
    c. If Data is not found in cache, it's called cache miss. Cache Library itself fetches the data from DB, stores it back to cache and data is returned to the application.



Pros:
1. Good approach for heavy read application.
2. The logic of fetching the data from DB and updating cache is separated from the application.

Cons:
1. For new data read, there will always be Cahce-Miss first.(to resolve this, generally we can pre-heat the cache)
2. Without appropriate caching is not used during write operation. There is a chance of inconsistency between Cache and DB.
3. Cache document structure should be same as DB table

3. **Write Around cache**:
    - Directly writes data into the DB.
    - It does not update the cache.



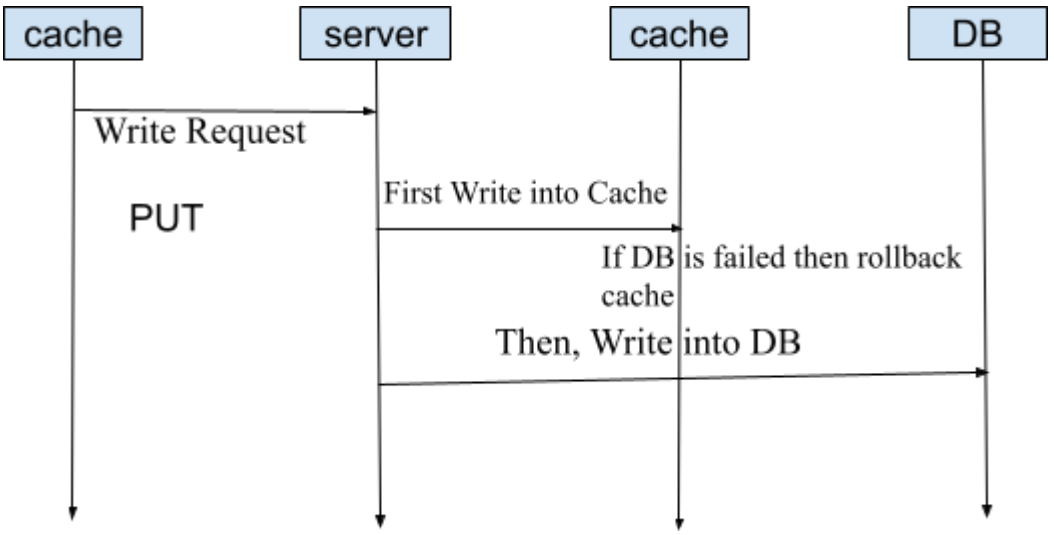
Pros:

- Good approach for Heavy Read application
- Resolves inconsistency problem between cache and DB.

Cons:

- For new data read, there will always be Cache-Miss first. (to resolve this, generally we can pre-heat the cache)
- If DB is down, the write operation will fail.

4. **Write through cache**:
    - First writes data into the cache.
    - Then in synchronous writes data into the DB.



Pros:

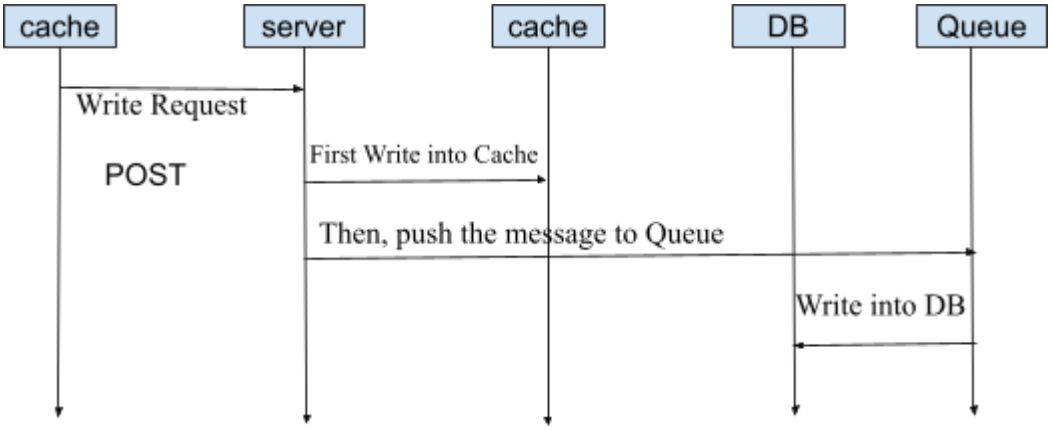
- Cache and DB always remain consistent.

- Cache Hit chance increases a lot.

Cons:

- Alone it is not useful, it will increase the latency. (That's why its always used with Read through or cache aside cache)
- 2 phase commit, need to be supported with this. To maintain the transactional property
- If DB is down, the write operation will fail.

5. **Write Back (or Behind) Cache**:
    - First writes data into the cache.
    - Then in Asynchronous writes data into the DB



Pros:

- Good for writing heavy applications.
- Improves the write operation latency. As writing into the DB happens asynchronously.
- Cache Hit chance increases a lot.
- Gives much better performance when used with read through cache.

Cons:

- Even when DB fails, the write operation will still work.
- If Data is removed from cache and DB write still not happens, then there is a chance of an issues.(it is handled by keeping the TAT of cache little higher like 2 days)

Distributed Transaction

Transaction Handling in Distributed System

What is Transaction?
It refers to a set of operations which need to be performed or simply a group of tasks which need to be performed against the DB.
It has 4 properties:

- **Atomicity**: All operations in a single transaction should be successful or all should fail.
- **Consistency:** DB should be in consistent state before and after the transaction.
- **Isolation**: More than one transaction that is running concurrently, appears to be serialized.
- **Durability**: After a transaction successfully completed, even if DB fails, data should not get lost.

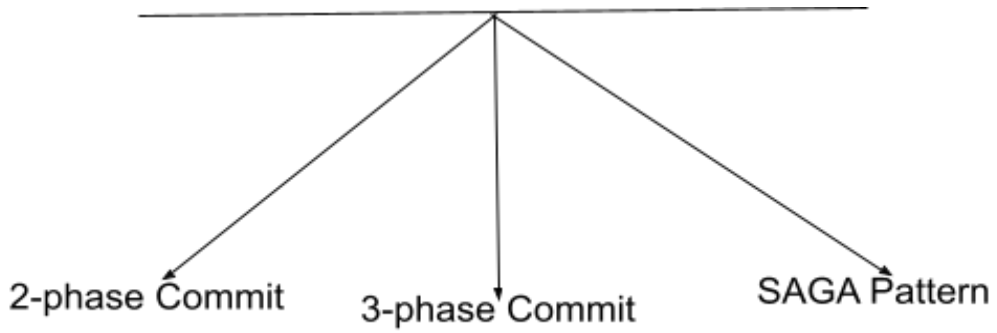
Example: below all operation will follow on one DB
Begin Transaction(T):

- Withdrawal Money from 'A': Rs 100
- Deposit Money to 'B': Rs 100

If all success:
    Do Commit
else
    Do Rollback

Close_Transaction;

How to handle this in a distributed system, where operations involve multiple databases?
OR
As Transaction is Local to a particular database? How will we satisfy the transaction property in a distributed system?



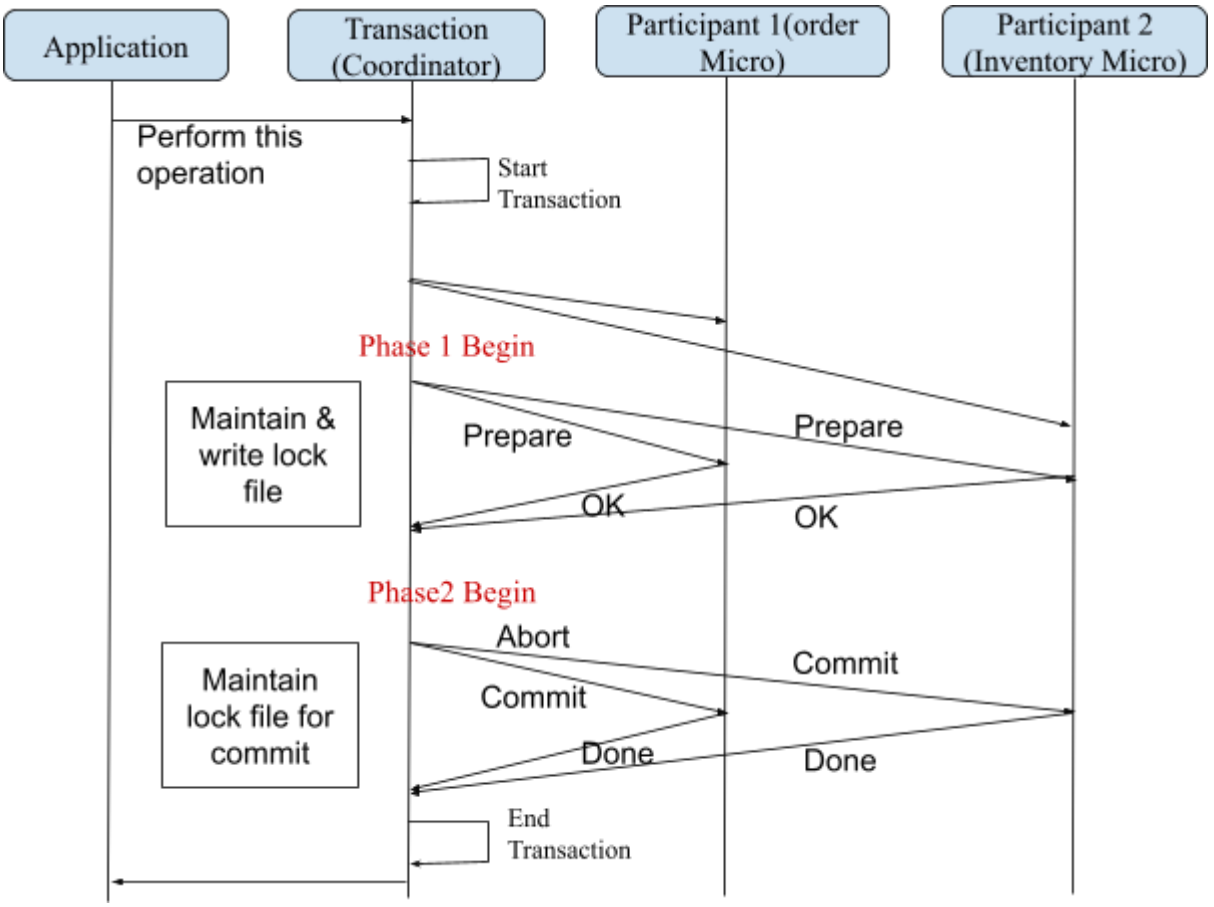
As name says, there are 2 phases in this protocol:

2-Phase Commit

1. Voting or Prepare phase
2. Decision or Commit Phase

Below diagram what issues will be come

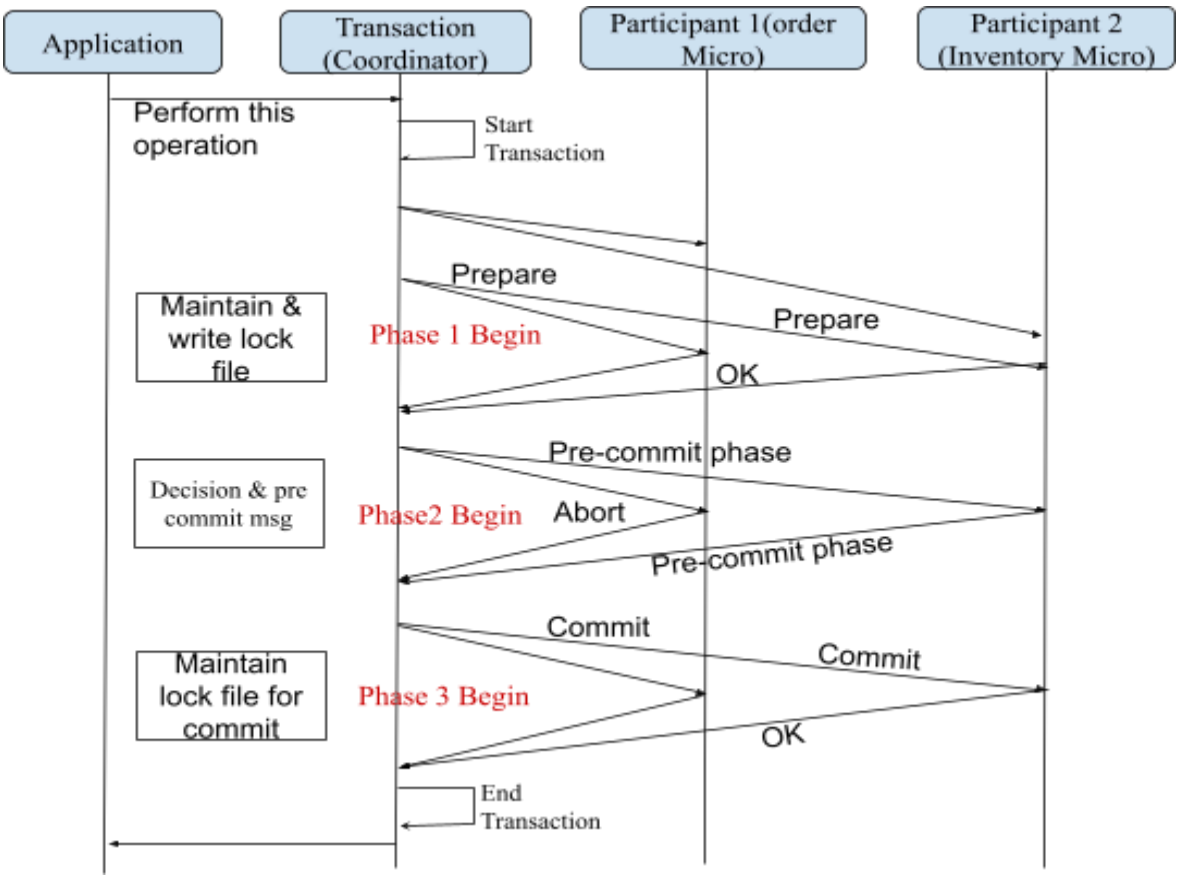
1. Transaction coordinator failed
2. Its is synchronous
3. Participants get failed
    a. Prepare msg lost
    b. OK msg lost
    c. Commit or abort lost



3-phase commit:(it's too much complicated)- Non blocking protocol
As name says, there are 2 phase in this protocol

1. Prepare phase
2. Pre-Commit phase
3. Commit Phase

If co-ordinator has blocked then it should not fail
Participant check with pre-commit if is there or not
Its is synchronous



SAGA Pattern

It is asynchronous

SAGA pattern is used when long trailing transaction (means so many participant involved)

It is sequential nature if any application invoking to other server

If anything failed in between then publish event or may be some queue msg failed according rollback transactions

SAGA is the distributed system with one transaction

Database Indexing

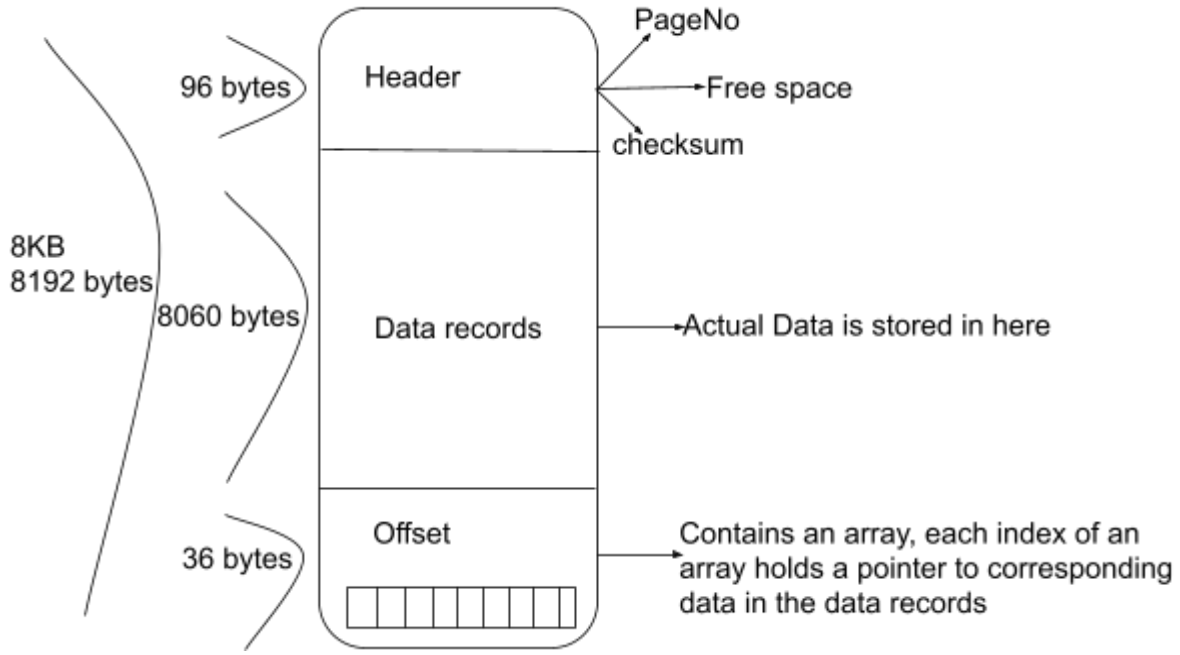
Database Indexing

How table data (rows) are actually stored?

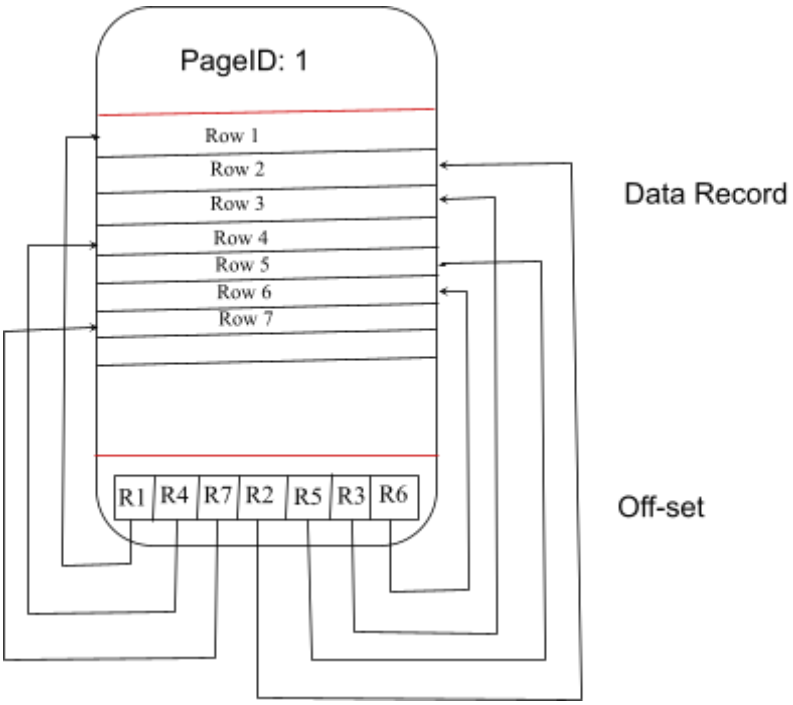
	Column1	Column2	Column3
	Emp ID	Name	Address
Row 1	1	A	City A
Row 2	2	B	City B
Row 3	3	C	City C
Row 4	4	D	City D

Above one is just a logical representation
Actual data is not stored in this way

- DBMS creates Data pages (generally its 8KB but depends upon DB to DB)
- Each Data page can store multiple table rows in it.



So if 1 table row is lets say 64bytes, so in 1 data page we can have ~125 DB rows
125 rows we can create it one table



DBMS creates and manages these data pages. As for storing 1 table data, it can create many data pages.
These data pages ultimately get stored in the **Data block** in physical memory like a disk.

What is a Data Block?

- Data Block is the minimum amount of data which can be read/write by an I/O operation.
- It is managed by an underlying storage system like a disk. Data Block size can range from 4kb to 32kb (common size is 8kb).
- So based on the data block size, it can hold 1 or many data pages.

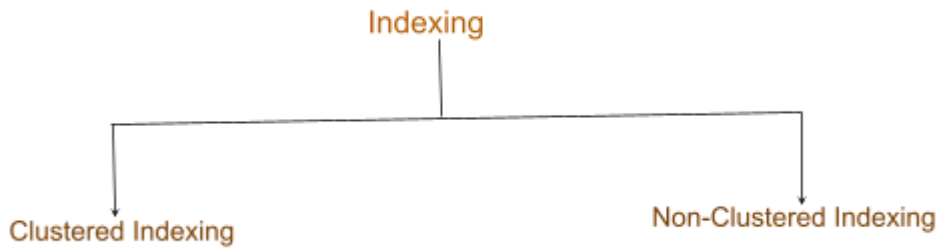
DBMS maintains the mapping of DataPage and Data Block.

Data Page1	Data Block 1
Data Page2	Data Block 1
Data Page3	Data Block 2
Data Page4	Data Block 2

Remember, DBMS controls Data pages (like what Row goes in which page or sequence of pages etc.) but has no control on Data blocks (data blocks can be scattered over the disk)

What is ordering if row set pointing to row
Ordering is not important role here

What type of indexing present?



Now before we understand clustered indexing and non-clustered indexing, we have to first understand,

What is Indexing?
What Data Structure it uses and how it works?

Indexing:

It is used to increase the performance of the database query. So that data can be fetched faster. Without indexing, DBMS has to iterate each and every table row to find the requested data I.e O(N), if there are millions of rows, query can take some time to fetch the data.

Which Data Structure provides better time complexity than O(N)?
B+ Tree provides O(log N) time complexity for insertion, searching & deletion.

How B(Balanced) + Tree works?

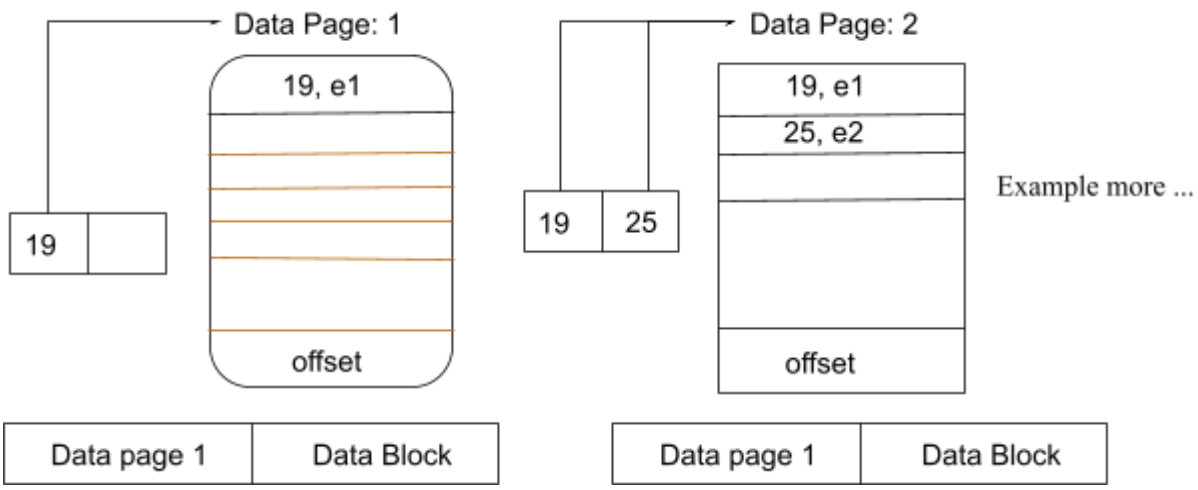
- It maintains sorted data.
- All leaf are at the same level
- M order B Tree means, each node can have at most M children.
- And M-1 keys per node.

B+ Tree is exactly the same as B tree, with additional features like child nodes are also linked to each other.

DBMS used B+ Tree to manage its **Data pages** and Rows within the pages
Root node or intermediary node hold the value which is used for faster searching the data. Possible that value might have been deleted from DB, but it can be used for sorting the tree. Leaf node actually holds the indexed column value.

Index column

EmpID	EmpName
19	e1
25	e2
30	e3
17	e4
6	e5

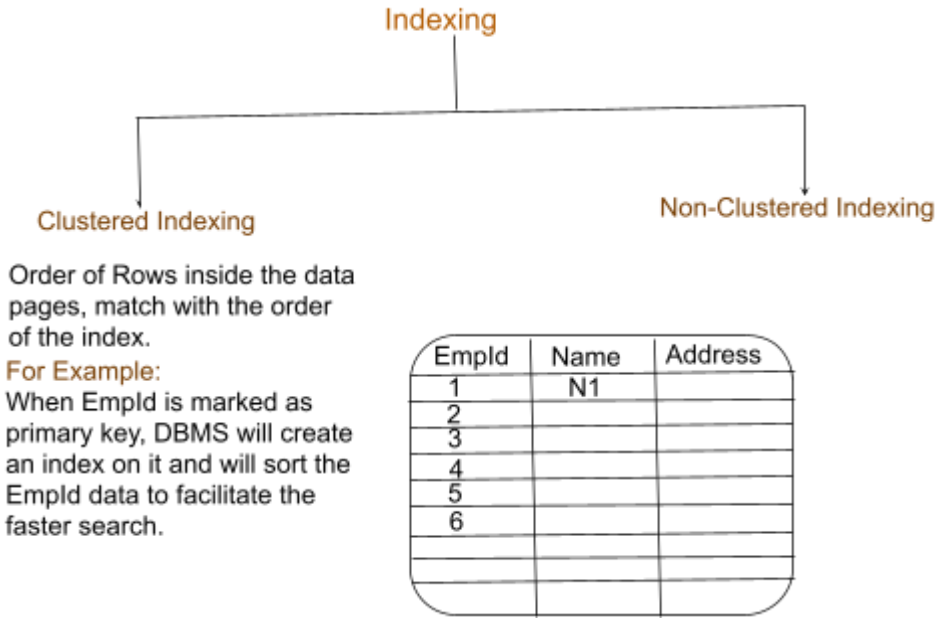


B+ Tree constructed

1 table can have 100 data pages
And each datapages store data block
DBMS maintain a mapping of

B+ tree data structure based DBMS will decide we will go which data block and data pages
DBMS will do page splitting and will update pointers if data pages are full.

Understanding the data structure used for indexing and how it works?



Offset maintain array where sorted indexes comes up

- There can be only 1 clustered index present/table because ordering of pages can be done based on one index only.

- If manually you have not specified any clustered index, then DBMS looks for Primary key which is unique and not null and uses it as a clustered key.
- If in table there is no primary key available, then internally it creates a hidden column which is used as a clustered index. (This columns just increase sequentially, so guaranteed unique and not null)
- Ordering offset doesn't depend on secondary index
- DBMS stores in index pages

1. Find the index page
2. B+ tree to find which data fetched
3. Datapage find the data block
4. Load data block in memory
5. Read the data

System Design: Distributed Concurrency Control

System Design: Distributed Concurrency Control

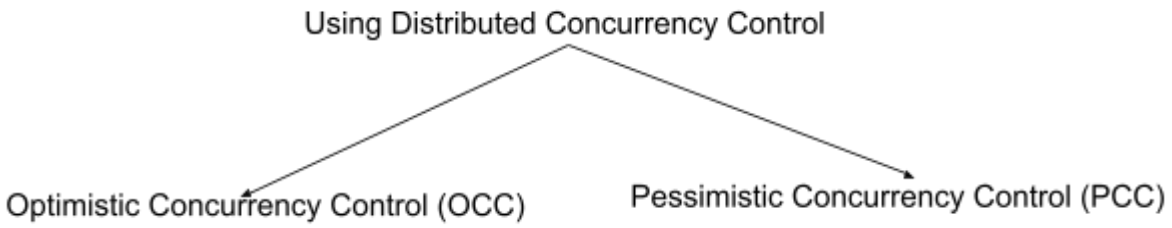
Lots of interview asked like
Low level design
Book my show
Parking lot

Scenario - Many concurrent request tries to book same movie theatre seat
Problems has come when multiple resources trying to access one request

In one application, synchronized block use for handle critical section

What is a process and thread and difference between ?
One process can have multiple threads.

Synchronized block it will not in distributed system



Before we need to understand below points

1. **What is the usage of Transaction**?
2. **What is DB Locking**?
3. **What is the Isolation level present**?

1. What is the usage of Transaction?
Transaction helps to achieve INTEGRITY. Means it helps us to avoid inconsistency in our database.

2. **What is DB Locking**?
DB locking, help us to make sure that no other transaction update the locked rows

Lock Type	Denoted By	Another Shared Lock	Another Exclusive Lock
Have Shared Lock	X- (W)	YES	NO
Have Exclusive Lock	S-(R)	NO	NO

Shared Lock generally known as read lock means read can happen and can't restrict any other transaction so if transaction has shared lock then not other transaction can take a exclusive lock

Exclusive lock: if any other transaction now can not even read it and can not even write it and exclusive lock generally use for write lock

Shared lock can put row on multiple transaction and can able to read it in parallel
Exclusive lock only one transaction can put no, once you put the exclusive lock no other transaction request for shared lock and no other transaction request for exclusive lock also

3. **What is the Isolation Level present**? - ACID - I - Isolation

Isolation Level	Dirty read possible	Non-Repeatable Read Possible	Phantom Read Possible
Read Uncommitted	YES	YES	YES
Read Committed	NO	YES	YES
Repeatable Read	NO	NO	YES
Serializable	NO	NO	No

What is Dirty Read?
If Transaction A is reading the data which is written by Transaction B and not yet even committed.
If Transaction B does the rollback, then whatever data read by Transaction A is know dirty read.

Timeline	Transaction A	Transaction B	DB
T1	BEGIN_TRANSACTION	BEGIN_TRANSACTION	ID:1, Status:Free
T2	Update Row ID:1, Status:Booked		ID:1, Status:Booked (Not Committed by Transaction A yet)
T3		Read Row ID:1 - (Got status as Booked)	ID:1, Status:Booked (Not Committed by Transaction A yet)
T4	Some Issue faced here	Some Computation	ID:1, Status:Booked (Not Committed by Transaction B yet)
T5	Rollback	Commit	ID:1, Status:Free

Non-Repeatable Read:

If suppose Transaction A, reads the same row several times and there is a chance that it reads different values.

Time series	Transaction A	DB
T1	BEGIN_TRANSACTION	ID:1 Status: Free
T2	Read Row ID: 1 (reads Status: Free)	ID: 1 Status : Free
T3		ID: 1 Status : Booked → Some other Transaction changed and committed the changes
T4	Read Row ID: 1 (reads Status: **Booked**)	ID: 1 Status:Booked
T5	Commit	

Phantom Read:

If suppose Transaction A, executes the same query several times and there is a chance that the rows returned are different.

Time Series	Transaction A	DB
T1	Begin_TRANSACTION	ID:1, Status:Free ID:3, Status:Booked
T2	Read Row where ID>0 and ID<5 (reads 2 rows ID: 1 and ID:3)	ID:1, Status:Free ID:3, Status:Booked
T3		ID:1, Status:Free ID:2, Status:Free ID:3, Status:Booked (Note: Some other transaction inserted the row with ID:2 and committed)
T4	Read Row where ID>0 and ID<5 (read 23rows ID:1, ID:2 and ID:3)	ID:1, Status:Free ID:2, Status:Free ID:3, Status:Booked
T5	Commit	

What is an isolation level and locking strategy?

Isolation Level	Locking Strategy
Read Uncommitted	Read: No Lock required Write: No Lock Required
Read committed	Read: Shared Lock acquired and Released as soon as Read is done Write: Exclusive Lock acquired and keep till the end of the transaction
Repeatable Read	Read: Shared Lock acquired and Released only at the end of the transaction Write: Exclusive Lock acquired and Released only at the end of the transaction
Serializable	Same as Repeatable Read Locking Strategy + apply Range Lock and lock is release only at the end of the transaction

```
SET TRANSACTION ISOLATION LEVEL REPEATABLE READ;

BEGIN_TRANSACTION;

Select Query;
Update query;
If Success:
        COMMIT TRANSACTION;
Else:
        ROLLBACK TRANSACTION;

END_TRANSACTION
```

Depends on which database you are using ISOLATION Level defined and if you are using in your project like @Transactional so its preferring default isolation level.

If you are implementing transaction then first you need to defined isolation layer and set isolation layer if you are not defining then will pick based on default DB wise

Optimistic concurrency control - it has allows used to have more concurrency and allow use to isolation level below above repeatable read or serializable isolation layer
Optimistic concurrency control focused on read uncommitted and read committed isolation layer and it is mostly use for read purpose

Pessimistic Concurrency control -
Mostly used for Repeatable Read and serializable isolation layer

Lets come back to the main solution of the problem
"**How to resolve concurrency issue**"

Using Distributed Concurrency control

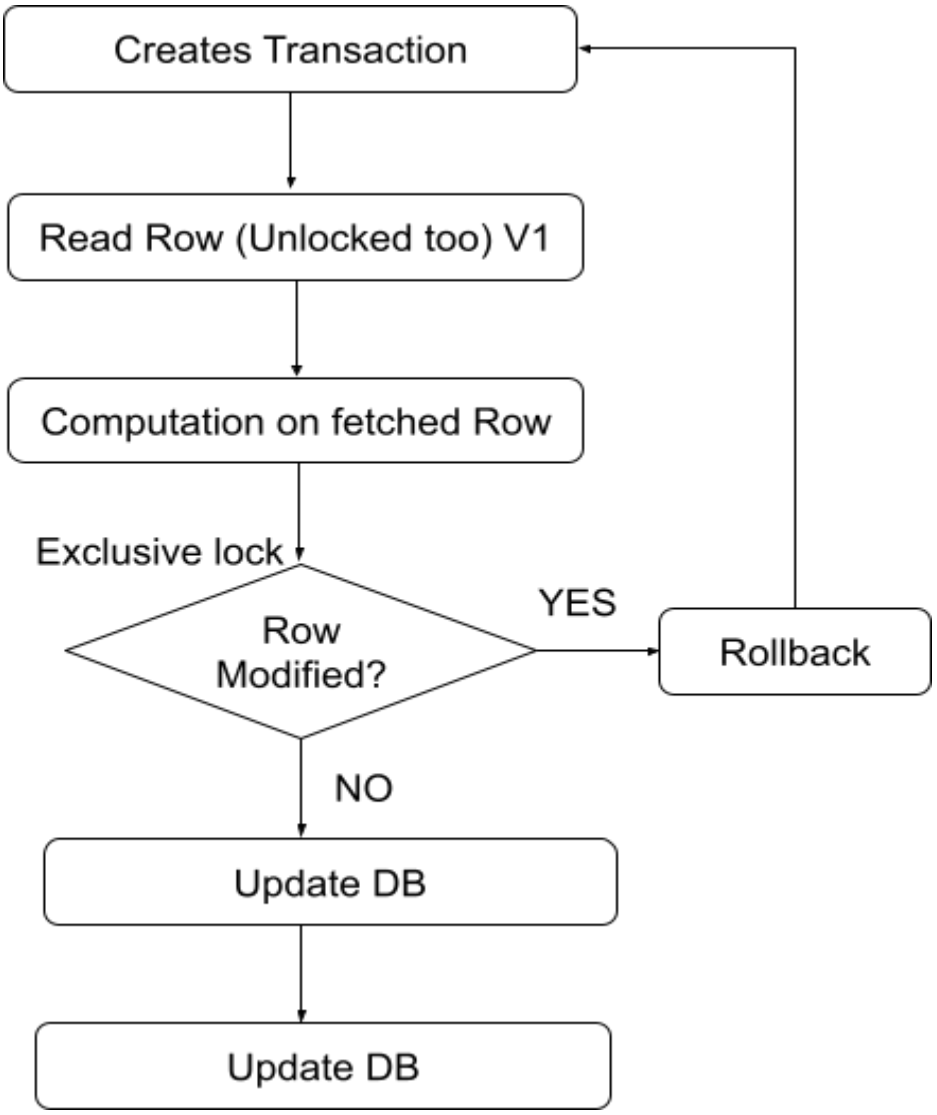
Optimistic Concurrency Control (OCC)
- Isolation level used below Repeatable read
- Much Higher Concurrency
- No Change of Deadlock
- In case of conflict, overhead of transaction rollback and retry logic is there
- optimistic use version and incrementally increase

Pessimistic Concurrency Control (OCC)
- Isolation level used repeatable read and serializable
- Less Concurrency compared to optimistic
- Deadlock is possible, then transaction stuck in deadlock forced to do rollback
- putting a long lock, sometimes timeout issue comes and need to be done.

Example **Optimistic Concurrency Control**

Transaction A	**Transaction B**	**DB**
BEGIN_TRANSACTION	BEGIN_TRANSACTION	ID:1,Status:Free,Version:1
Read Row ID:1 (Row Version is 1)	Read Row ID:1 (Row version is 1)	ID:1,Status:Free,Version:1
Select for Update (version Validation happens)		ID:1, Status:Free,Version:1 (Exclusive Lock by TXN A)
Update Row ID:1, Status:Booked, Version:2		ID:1,Status:Booked,Version:2 (Exclusive Lock by TXN A)
Commit		ID:1, Status:Booked,Version:2 (Remove Exclusive lock from txn A)
	Status for update (version validation happens)	ID:1,Status:Booked,Version:2 (Exclusive Lock by TXN B)
	Rollback	ID:1,Status:Booked,Version:2 (Remove exclusive lock by txn B

Optimistic Concurrency flow



If currency suddenly comes up then its is best optimistic concurrency control

Pessimistic Concurrency flow

Pessimistic concurrency comes with serializable (Sequence wise no matter who has started first)
Serializable means once A has completed then comes up B till time B will wait and no body can read it data also (shared resources)
There is one big problem dead-lock in pessimistic concurrency flow
A **deadlock** in Java occurs when two or more threads are blocked forever, each waiting for the other to release a resource.

Waiting for T1 release shared lock on A and now both are waiting they won't transaction to complete transaction means both are waiting for each other

If a transaction has stuck then aborted after a particular time and then we need to restart the transaction.

Optimistic hasn't problem for deadlock but **pessimistic** concurrency has problem for deadlock

Pessimistic concurrency control has **2 Phase locking**

If you want to decide which one we need to use so first identify the isolation level you might require then you can choose optimistic /pessimistic concurrency control. Optimistic not only good at every time depending on requirement or what you are resolving issues.

Two Phase Locking

Two Phase Locking

What is two phase Locking?
It is a concurrency control protocol used in database systems and transaction management to ensure serializability, which is the highest level of consistency in a transaction schedule. It ensures that transactions are executed in a way that they appear to be serial (executed one after another) even when they are executed concurrently.

Basic Two Phase locking

Phase 1: Growing Phase

Phase 2: Shrinking Phase